COMPTIA SECURITY+ SY0-701 | DOMÍNIO 2 · AMEAÇAS, VULNERABILIDADES E MITIGAÇÕES (22%)

# Security+ 2.4 · Application Attacks

Mestre CyberSec · **Aula 2.4 — Application Attacks** (11:48) · Assistir Vídeo no YouTube ↗ · Treino Interativo: edusouzamarques.github.io/cybersec

**Resumo Executivo da Aula:** Aplicação = a superfície mais atacada: privilege escalation (horizontal = mesmo nível, outro dado; vertical = tier acima — user vira admin); directory traversal (../../etc/passwd); remote code execution. Defesa: input validation, patching, WAF, SCA (library vulnerable), secure co

## 💡 Highlights que a prova CompTIA cobra

- Privesc horizontal (mesmo nível, outro dado) vs vertical (tier acima)
- Directory traversal: sobe com ../ pra /etc/passwd
- Defesa: input validation SEMPRE; nunca confie no input

## 🎯 Dicas de Prova & Cenários Típicos da Banca

- 'user gains higher-level access' = privilege escalation (vertical)
- 'path traversal reads arbitrary files' = directory traversal

## 🚨 Além do Vídeo: Conceitos Oficiais, Siglas e Armadilhas (Seção 2.4 — Application Attacks)

**Por que este aprofundamento é cobrado na prova?:** O vídeo resume os tópicos práticos, mas o exame CompTIA SY0-701 cobra discernimento técnico rigoroso sobre as tecnologias e protocolos específicos desta seção:

**• Escalação de Privilégio (PE)**

Ataque em que um usuário obtém níveis de acesso não autorizados em um sistema. A escalação vertical sobe para um nível de privilégio maior (ex.: usuário para admin); a horizontal acessa outra conta no mesmo nível de privilégio.

⚠️ ***Armadilha CompTIA:*** *If the question says 'user gains higher-level access' or 'user becomes admin,' it is vertical privilege escalation. Horizontal = same level, different data/account.*

**• Traversal de Diretório (DT)**

Ataque de injeção que usa sequências como '../' (ponto-ponto-barra) em um caminho de arquivo para escapar do diretório pretendido e ler arquivos arbitrários no servidor, como /etc/passwd ou arquivos de configuração.

⚠️ ***Armadilha CompTIA:*** *Look for clues like '../', 'path traversal', or 'arbitrary file read.' The classic target file on Linux is /etc/passwd. Mitigation = input validation blocking '../' sequences.*

**• Execução Remota de Código (RCE)**

Vulnerabilidade crítica em que o atacante executa remotamente comandos ou código arbitrário no sistema alvo, geralmente por meio de falhas de injeção, bugs de deserialização ou serviços sem patch.

⚠️ ***Armadilha CompTIA:*** *RCE is the highest-severity outcome. Watch for terms 'arbitrary code execution,' 'shell access,' or 'command injection' leading to full system compromise.*

**• Validação de Entrada (IV)**

Prática de codificação segura que verifica, sanitiza e restringe toda a entrada fornecida pelo usuário contra uma allowlist de valores esperados antes do processamento, prevenindo ataques de injeção e traversal.

⚠️ ***Armadilha CompTIA:*** *The golden rule: 'Never trust user input.' Input validation is the first-line defense against SQLi, XSS, directory traversal, and command injection — expect it as the answer whenever these attacks appear.*

**• Firewall de Aplicação Web (WAF)**

Firewall especializado de Camada 7 que inspeciona tráfego HTTP/HTTPS para detectar e bloquear ataques web como SQL injection, XSS e traversal de diretório contra aplicações web.

⚠️ ***Armadilha CompTIA:*** *Distinguish WAF (Layer 7, protects web apps from HTTP-layer attacks) from NGFW (broader Layer 3-7 network protection). WAF = OWASP Top 10 shield.*

**• Análise de Composição de Software (SCA)**
Ferramenta de segurança que escaneia bibliotecas de terceiros, dependências e componentes open-source da aplicação contra bases de vulnerabilidades para identificar CVEs conhecidos e problemas de licenciamento.

⚠️ ***Armadilha CompTIA:*** *SCA = third-party/library scanning (Log4Shell-style vulnerabilities). Do not confuse with SAST (scans YOUR source code) or DAST (runtime black-box testing).*

**• Injeção de SQL (SQLi)**
Ataque de injeção em que instruções SQL maliciosas são inseridas em um campo de entrada para manipular o banco de dados backend, podendo expor, alterar ou apagar dados.

⚠️ ***Armadilha CompTIA:*** *Always pair SQLi mitigation with 'parameterized queries / prepared statements.' Input validation alone is NOT enough — keywords like 'UNION SELECT' or 'OR 1=1' are red flags.*

**• Cross-Site Scripting (XSS)**
Falha de injeção em que scripts maliciosos são injetados em uma página web confiável e executados no navegador da vítima. XSS stored persiste no servidor; reflected é entregue via link crafted.

⚠️ ***Armadilha CompTIA:*** *XSS targets the user's browser/client (not the server). Mitigation = output encoding and Content Security Policy (CSP). If the question mentions 'stealing cookies' or 'session hijacking via script,' think XSS.*

## 🏛️ Visão Didática, Analogias & Análise de Governança (GRC)

### 📌 SSRF (Server-Side Request Forgery)

🏠 **Analogia Intuitiva:** É como você ir até o guichê de atendimento de um prédio governamental ultra-protegido e entregar um bilhete para o mensageiro oficial dizendo: 'Por favor, vá até a sala secreta dos arquivos presidenciais no 10º andar, tire uma foto do documento confidencial e traga aqui para eu ver na recepção'. Como o mensageiro é um funcionário credenciado da casa que tem crachá de livre acesso a todos os setores internos (onde você, como cidadão comum da rua, é barrado pelos seguranças), ele obedece sem desconfiar, busca o segredo confidencial na rede interna restrita e entrega para você no balcão da calçada.

🔍 **Causa Raiz / Por que foi criado:** Aplicações web frequentemente precisam buscar recursos externos (como importar imagens via URL ou consultar webhooks). Se a aplicação não validar rigorosamente o endereço de destino fornecido pelo usuário, o invasor submete URLs apontando para a rede interna ou loopback (`http://127.0.0.1`, `http://localhost`) ou para o serviço de metadados da nuvem (`http://169.254.169.254/latest/meta-data/`), roubando credenciais de IAM e segredos corporativos.

🛡️ **Visão do Auditor de GRC:** Auditar a aplicação de listas de permissão (allowlists) de domínios externos; desabilitar resolução de esquemas não web (como `file://` ou `gopher://`); bloquear resoluções para faixas IP privadas na camada de aplicação; e exigir a ativação do IMDSv2 em instâncias de nuvem (que exige cabeçalho de token de sessão HTTP PUT para neutralizar SSRF).

⚠️ **Como cai na Prova CompTIA:** Trigger: 'attacker tricks a public web server into executing requests against internal network resources or the cloud metadata endpoint 169.254.169.254' -> SSRF (Server-Side Request Forgery).

### 📌 CSRF (Cross-Site Request Forgery)

🏠 **Analogia Intuitiva:** Você já está com a sua sessão aberta no site do banco no navegador. Em outra aba, você clica em um link de meme ou promoção falsa que um desconhecido te enviou. A página fraudulenta contém um código invisível que manda um pedido para o seu banco: 'Transfira R$ 2.000 para a conta do golpista'. O seu navegador envia o pedido anexando automaticamente os seus cookies de login legítimos. Como o banco reconhece seu cookie de sessão autêntico, ele acha que foi você quem ordenou a transferência e autoriza a operação sem você saber.

🔍 **Causa Raiz / Por que foi criado:** Navegadores web automaticamente incluem cookies armazenados de autenticação em todas as requisições enviadas ao domínio de destino, mesmo quando o disparo é originado por páginas de terceiros. Sem validação de intenção através de tokens exclusivos por transação, o servidor não consegue distinguir requisições voluntárias de requisições forjadas.

🛡️ **Visão do Auditor de GRC:** Implementar tokens anti-CSRF aleatórios e imprevisíveis (Synchronizer Token Pattern) em formulários com alteração de estado; configurar o atributo `SameSite=Strict` ou `SameSite=Lax` em todos os cookies de sessão; e exigir reautenticação para transações financeiras críticas.

⚠️ **Como cai na Prova CompTIA:** Trigger: 'attacker tricks a victim's authenticated browser into executing unwanted actions on a trusted web application' -> Cross-Site Request Forgery (CSRF). Mitigações padrão: Anti-CSRF Tokens e atributo de cookie SameSite.

### 📌 BOLA (Broken Object Level Authorization) e Falhas de API

🏠 **Analogia Intuitiva:** BOLA (antigo IDOR) é como você ir ao armário de guarda-volumes da academia com a chave do armário número 101. Ao terminar o treino, você olha para o armário do lado, vê a fechadura do armário 102 e simplesmente gira a maçaneta ou usa um clipe de papel; a porta do armário 102 se abre imediatamente e você vê as roupas e carteira de outro aluno. O sistema conferiu que você era um aluno matriculado da academia (autenticação válida), mas não checou se você tinha autorização para abrir ESPECIFICAMENTE o armário 102.

🔍 **Causa Raiz / Por que foi criado:** Em APIs REST modernas, o servidor recebe requisições com IDs de objetos fornecidos pelo cliente (ex: `/api/v1/invoices?id=5002`). A aplicação confere se o token de acesso do usuário é válido (autenticação), mas não valida se o usuário autenticado possui o direito de posse/leitura daquele objeto específico 5002 (autorização a nível de objeto). BOLA é a falha nº 1 do OWASP API Top 10.

🛡️ **Visão do Auditor de GRC:** Implementar controle de acesso baseado em atributos (ABAC) validando a posse do objeto no banco de dados para cada requisição; substituir IDs numéricos incrementais sequenciais por identificadores opacos e imprevisíveis (UUIDv4); e utilizar API Gateways para fiscalização de esquemas de autorização.

⚠️ **Como cai na Prova CompTIA:** Trigger: 'user modifies an account number or invoice ID in an API request (e.g., from id=101 to id=102) and successfully accesses another client's records'. Resposta: Broken Object Level Authorization (BOLA) ou IDOR.

### 📌 Poluição de Parâmetros HTTP (HTTP Parameter Pollution - HPP)

🏠 **Analogia Intuitiva:** É como preencher um formulário impresso de pagamento que tem o campo 'Favorecido:' e você escrever duas vezes no formulário: na primeira linha você escreve o nome da sua empresa e na segunda linha escreve o nome de um criminoso. O auditor do portão de entrada (WAF) olha apenas para a primeira linha, acha legal e aprova; mas o caixa do cofre no porão (servidor backend) olha apenas para a segunda linha e transfere todo o dinheiro para o criminoso. A divergência de interpretação entre quem fiscaliza e quem executa cria a brecha.

🔍 **Causa Raiz / Por que foi criado:** Diferentes servidores e tecnologias web lidam com parâmetros duplicados de maneiras conflitantes (alguns consideram o primeiro, outros consideram o último, outros concatenam com vírgula). Invasores exploram essa discrepância enviando requisições com parâmetros repetidos (`?user=guest&user=admin`) para burlar filtros de segurança do WAF e escalar privilégios no backend.

🛡️ **Visão do Auditor de GRC:** Padronizar o tratamento de parâmetros em toda a pilha arquitetural; rejeitar requisições HTTP que contenham nomes de parâmetros duplicados no cabeçalho ou corpo; e validar entradas contra especificações OpenAPI estritas.

⚠️ **Como cai na Prova CompTIA:** Trigger: 'attacker supplies multiple HTTP parameters with the same name to bypass perimeter validation filters due to differences in parsing logic between proxy and application' -> HTTP Parameter Pollution (HPP).

## ⚡ TABELA DE FIXAÇÃO RÁPIDA PARA O EXAME (CHEAT SHEET)

| Cenário Típico de Exame / Situação Prática | Conceito Técnico & Resposta da Banca |
|---|---|
| Atacante induz o servidor web a realizar requisições não autorizadas para recursos internos ou para o IP de metadados da nuvem (169.254.169.254) | **Server-Side Request Forgery (SSRF)** |
| Página maliciosa de terceiros induz o navegador autenticado do usuário a submeter transações indesejadas em uma aplicação confiável | **Cross-Site Request Forgery (CSRF)** |
| Usuário autenticado altera o parâmetro de ID em uma chamada de API (ex: id=101 para 102) e visualiza dados privados de outro cliente | **Broken Object Level Authorization (BOLA) / IDOR** |
| Atacante envia múltiplos parâmetros com o mesmo nome para explorar divergências de interpretação entre o WAF e o backend | **Poluição de Parâmetros HTTP (Parameter Pollution / HPP)** |
| Defesa web que anexa um token criptográfico descartável a cada formulário para impedir que ações forjadas sejam aceitas | **Token Anti-CSRF (Synchronizer Token)** |
| Versão do serviço de metadados da AWS que exige cabeçalho de token de sessão HTTP PUT, mitigando ataques de SSRF | **IMDSv2 (Instance Metadata Service Version 2)** |

## ✅ Perguntas de Treino Comentadas (10 Questões de Cenários Reais)

Cenários autênticos alinhados aos objetivos do exame. O gabarito com análise completa dos distratores encontra-se ao final.

**QUESTÃO 1 · CENÁRIO DE EXAME**

**Um analista de segurança revisa logs de servidor web e vê solicitações repetidas de URLs como /download?file=../../../../etc/passwd e /download?file=..%2f..%2fconfig.php. A aplicação está retornando conteúdo de arquivos fora do diretório pretendido. Qual ataque de aplicação está ocorrendo?**

**A)** Injeção de SQL

**B)** Cross-site scripting

**C)** Directory traversal

**D)** Escalação de privilégios

**QUESTÃO 2 · CENÁRIO DE EXAME**

**Uma aplicação web reflete o valor de um parâmetro de busca diretamente na resposta HTML sem codificação. Um atacante envia um termo de busca contendo uma tag de script maliciosa, e o script é executado no navegador de outros usuários quando eles visualizam os resultados da busca. Qual vulnerabilidade está sendo explorada?**

**A)** Cross-site scripting refletido

**B)** Injeção de SQL

**C)** Directory traversal

**D)** Execução remota de código

**QUESTÃO 3 · CENÁRIO DE EXAME**

**Um formulário de login monta uma consulta ao banco de dados concatenando os campos de nome de usuário e senha diretamente na instrução SQL. Um atacante insere um nome de usuário especialmente criado que altera a lógica da consulta e contorna a autenticação. Qual ataque é este?**

**A)** Cross-site request forgery

**B)** Injeção de SQL

**C)** Directory traversal

**D)** Cross-site scripting

**QUESTÃO 4 · CENÁRIO DE EXAME**

**Um usuário autenticado em uma aplicação bancária online altera o parâmetro de ID de conta em uma requisição do próprio número de conta para o número de conta de outro cliente. A aplicação retorna os detalhes da conta do outro cliente, embora ambos tenham o mesmo papel. Que tipo de escalação de privilégios é esta?**

**A)** Escalação de privilégios vertical

**B)** Escalação de privilégios horizontal

**C)** Directory traversal

**D)** Execução remota de código

**QUESTÃO 5 · CENÁRIO DE EXAME**

**Um atacante com uma conta de usuário padrão manipula um parâmetro oculto em uma requisição de atualização de perfil. A aplicação então concede à conta acesso a funções administrativas normalmente restritas à equipe de TI. Que tipo de escalação de privilégios ocorreu?**

**A)** Escalação de privilégios horizontal

**B)** Escalação de privilégios vertical

**C)** Injeção de SQL

**D)** Cross-site scripting

**QUESTÃO 6 · CENÁRIO DE EXAME**

**Uma aplicação web permite que usuários façam upload de arquivos de imagem. Após o upload, o servidor processa os metadados usando uma biblioteca que invoca um comando externo. Um atacante embute comandos de shell nos metadados, e o servidor os executa com as permissões da conta do serviço web. Qual vulnerabilidade foi explorada?**

**A)** Cross-site scripting

**B)** Directory traversal

**C)** Execução remota de código

**D)** Análise de composição de software

**QUESTÃO 7 · CENÁRIO DE EXAME**

**Uma equipe de desenvolvimento deseja identificar vulnerabilidades conhecidas em bibliotecas e frameworks de código aberto usados por suas aplicações. A equipe precisa de uma ferramenta que faça inventário de dependências e as relacione a CVEs conhecidas. Que tipo de análise deve ser usada?**

**A)** Teste dinâmico de segurança de aplicações

**B)** Análise de composição de software

**C)** Firewall de aplicação web

**D)** Teste estático de segurança de aplicações

**QUESTÃO 8 · CENÁRIO DE EXAME**

**Uma organização deseja proteger uma aplicação web pública contra ataques comuns de injeção antes que as requisições cheguem aos servidores de aplicação. A equipe de segurança precisa de um controle que inspecione o tráfego HTTP e possa bloquear padrões maliciosos. Qual controle atende melhor a esse requisito?**

**A)** Análise de composição de software

**B)** Firewall de aplicação web

**C)** Validação de entrada

**D)** Gestão de patches

**QUESTÃO 9 · CENÁRIO DE EXAME**

**Durante uma revisão de código, um engenheiro de segurança descobre que um formulário web depende apenas de verificações JavaScript no navegador para validar os campos de idade e código postal. Atacantes podem contornar essas verificações e enviar valores maliciosos diretamente ao servidor. Qual defesa deve ser adicionada?**

**A)** Firewall de aplicação web

**B)** Validação de entrada no lado do servidor

**C)** Análise de composição de software

**D)** Prevenção de directory traversal

**QUESTÃO 10 · CENÁRIO DE EXAME**

**Um recurso de download de arquivos constrói um caminho de arquivo a partir de um parâmetro fornecido pelo usuário. Os desenvolvedores querem impedir que atacantes acessem arquivos fora do diretório de download pretendido. Qual controle deve ser implementado?**

**A)** Codificar toda a saída HTML antes de renderizar páginas

**B)** Canonicalizar o caminho e validá-lo contra uma allowlist ou diretório base

**C)** Usar consultas de banco de dados parametrizadas para todas as buscas de metadados de arquivos

**D)** Executar teste dinâmico de segurança de aplicações no ambiente de homologação

## GABARITO COMENTADO COM ANÁLISE DE DISTRATORES

**Questão 1: Alternativa C — Directory traversal**

Directory traversal usa sequências ../ ou variantes codificadas para sair do diretório pretendido e acessar arquivos fora da raiz web. SQLi ataca consultas de banco de dados, XSS ataca execução no navegador, e escalação de privilégios obtém direitos superiores; a manipulação de caminho de arquivo é o indicador principal.

**Questão 2: Alternativa A — Cross-site scripting refletido**

XSS refletido ocorre quando a entrada do usuário é retornada imediatamente na resposta sem codificação de saída, permitindo que scripts sejam executados no navegador da vítima. SQLi afeta consultas ao banco de dados, directory traversal acessa arquivos, e RCE executa código no servidor.

**Questão 3: Alternativa B — Injeção de SQL**

Injeção de SQL ocorre quando entrada não confiável altera a estrutura de uma consulta SQL. Neste caso, o nome de usuário criado altera a condição da consulta para contornar a autenticação. CSRF abusa de sessões autenticadas, directory traversal manipula caminhos de arquivo, e XSS executa scripts em navegadores.

**Questão 4: Alternativa B — Escalação de privilégios horizontal**

Escalação de privilégios horizontal ocorre quando um usuário acessa recursos pertencentes a outro usuário no mesmo nível de privilégio. A escalação vertical envolveria obter privilégios superiores, como se tornar administrador.

**Questão 5: Alternativa B — Escalação de privilégios vertical**

Escalação de privilégios vertical ocorre quando um usuário obtém um nível de acesso superior, como passar de usuário padrão para administrador. A escalação horizontal envolveria acessar recursos de outro usuário no mesmo nível.

**Questão 6: Alternativa C — Execução remota de código**

Execução remota de código permite que um atacante execute comandos ou código arbitrários no servidor. Os comandos de shell embutidos executados pelo servidor indicam RCE. XSS é executado no navegador, directory traversal lê arquivos, e SCA é um processo de análise de dependências, não um ataque.

**Questão 7: Alternativa B — Análise de composição de software**

Análise de composição de software (SCA) examina componentes e dependências de terceiros para identificar vulnerabilidades conhecidas e questões de licença. DAST testa aplicações em execução, SAST analisa código-fonte, e um WAF bloqueia tráfego em tempo de execução.

**Questão 8: Alternativa B — Firewall de aplicação web**

Um firewall de aplicação web (WAF) inspeciona tráfego HTTP/HTTPS e pode bloquear ataques comuns como injeção de SQL e XSS antes que cheguem à aplicação. SCA analisa dependências, validação de entrada é implementada na aplicação, e gestão de patches corrige falhas conhecidas de software.

**Questão 9: Alternativa B — Validação de entrada no lado do servidor**

Validação de entrada no lado do servidor garante que os dados sejam validados no servidor independentemente das verificações do lado do cliente, que podem ser contornadas. Um WAF pode ajudar, mas não substitui o tratamento seguro de entrada; SCA e prevenção de directory traversal tratam de outros problemas.

**Questão 10: Alternativa B — Canonicalizar o caminho e validá-lo contra uma allowlist ou diretório base**

Canonicalizar o caminho e validá-lo contra uma allowlist ou diretório base impede directory traversal ao resolver sequências de caminho relativas e codificadas antes do acesso ao arquivo. Codificação de saída trata XSS, consultas parametrizadas tratam injeção de SQL, e DAST é um método de teste, não um controle preventivo.